THIRD YEAR COMPUTER ENGINEERING — ARTIFICIAL INTELLIGENCE LAB

Use DFS for problem solving. Examples can be water jug problem , missionaries and cannibals , maze solving, blocks world etc.

Academic Practical Manual & Write-up

1. TITLE & AIM

Title: Use DFS for problem solving. Examples can be water jug problem , missionaries and cannibals , maze solving, blocks world etc.

Aim: To implement Depth-First Search (DFS) algorithm in Python for solving state-space search problems (exemplified by the Water Jug Problem and applicable to Missionaries & Cannibals, Maze Solving, and Blocks World) to find a path from an initial state to a goal state.

2. THEORY

Depth-First Search (DFS) is an uninformed (blind) graph search algorithm that explores as deep as possible along each branch before backtracking. It utilizes a **Last-In, First-Out (LIFO)** data structure, typically implemented via a **Stack** or through **Recursion**.

Key concepts include:

- **State-Space Graph:** A set of discrete states representing configurations of the problem environment, connected by legal transition actions.
- **Initial State (S_0):** Starting node or initial configuration of the problem.
- **Goal Test:** A predicate function evaluating whether a given state satisfies the target conditions.
- **LIFO Traversal:** Expands the deepest unvisited node in the current frontier before exploring alternative siblings.

3. REQUIREMENTS

Component	Specification / Tools
Hardware	Standard PC / Laptop (Intel i3/AMD Ryzen 3 or higher, 4 GB RAM minimum)
Software	Python 3.x Environment
IDE / Tools	VS Code, Jupyter Notebook, PyCharm, or Python IDLE

4. PROCEDURE

1. Formulate the state representation (e.g., a tuple `(jug1, jug2)` for the Water Jug problem).
2. Define the initial state and the goal condition predicate.
3. Construct a successor function that generates all legal transition states from a current state.
4. Maintain a **Stack** (`frontier`) to keep track of nodes to explore and a **Set** (`visited`) to avoid infinite loops and repeated states.
5. Pop the top state from the stack:
   - If it satisfies the goal test, reconstruct and return the complete solution path.
   - Otherwise, mark it as visited, generate its legal unvisited successors, push them onto the stack, and repeat.
6. Print the ordered sequence of state transitions leading from start to goal.

5. IMPLEMENTATION CODE

The source code below is styled with blue line highlights:

```
# Problem: Water Jug Problem using DFS
# Capacity: Jug A = 4 Liters, Jug B = 3 Liters
# Goal: Measure exactly 2 Liters in Jug A

JUG_A_CAP = 4
JUG_B_CAP = 3
GOAL_A = 2

def get_successors(state):
    a, b = state
    successors = []

    # 1. Fill Jug A
    successors.append((JUG_A_CAP, b))
    # 2. Fill Jug B
    successors.append((a, JUG_B_CAP))
    # 3. Empty Jug A
    successors.append((0, b))
    # 4. Empty Jug B
    successors.append((a, 0))
    # 5. Pour A -> B
    pour_a_b = min(a, JUG_B_CAP - b)
    successors.append((a - pour_a_b, b + pour_a_b))
    # 6. Pour B -> A
    pour_b_a = min(b, JUG_A_CAP - a)
    successors.append((a + pour_b_a, b - pour_b_a))

    return successors

def dfs_water_jug(start_state):
    # Stack stores tuples of (current_state, path_taken)
    stack = [(start_state, [start_state])]
    visited = set()

    while stack:
```

```
        current_state, path = stack.pop()

        a, b = current_state
        # Goal Condition: Jug A contains exactly 2 Liters
        if a == GOAL_A:
            return path

        if current_state not in visited:
            visited.add(current_state)

            # Generate and push valid unvisited successors
            for successor in get_successors(current_state):
                if successor not in visited:
                    stack.append((successor, path + [successor]))

    return None

if __name__ == "__main__":
    initial_state = (0, 0)
    solution_path = dfs_water_jug(initial_state)

    print("--------------------------------------------------")
    print("DFS Water Jug Problem Solution Path:")
    print("--------------------------------------------------")
    if solution_path:
        for step, state in enumerate(solution_path):
            print(f"Step {step}: Jug A = {state[0]}L, Jug B = {state[1]}L")
    else:
        print("No solution found.")
```

6. OUTPUT

```
--------------------------------------------------
DFS Water Jug Problem Solution Path:
--------------------------------------------------
Step 0: Jug A = 0L, Jug B = 0L
Step 1: Jug A = 0L, Jug B = 3L
Step 2: Jug A = 3L, Jug B = 0L
Step 3: Jug A = 3L, Jug B = 3L
Step 4: Jug A = 4L, Jug B = 2L
Step 5: Jug A = 0L, Jug B = 2L
Step 6: Jug A = 2L, Jug B = 0L
```

7. ANALYSIS AND EVALUATION

- **Time Complexity:** $O(b^m)$, where b is the maximum branching factor and m is the maximum search depth.
- **Space Complexity:** $O(b \times m)$, due to storing nodes along the current search path in memory (substantially lower memory usage than BFS).
- **Completeness:** DFS is **not complete** in infinite state spaces unless loop detection (visited tracking) or depth limits are imposed.

- **Optimality:** DFS is **not optimal**; it returns the first path found to the target, which may contain redundant steps.

8. CONCLUSION

Depth-First Search was successfully implemented in Python to solve state-space search problems using the Water Jug problem as a benchmark. The practical demonstrates how stack-driven exploration navigates complex decision trees efficiently in memory, making it well-suited for deep trees where any valid goal state is acceptable.

9. VIVA QUESTIONS & ANSWERS

Q1: How does DFS differ from BFS in terms of data structures and memory usage?
Answer: DFS uses a **Stack (LIFO)** and consumes linear memory $O(b \times m)$, whereas BFS uses a **Queue (FIFO)** and consumes exponential memory $O(b^d)$ to track entire search levels.

Q2: Why is DFS non-optimal?
Answer: DFS explores one branch down to its maximum depth before evaluating alternative branches. It terminates at the first goal node encountered, which is not guaranteed to be the shortest path.

Q3: How does DFS handle cyclic loops in graph search problems like Maze Solving or Water Jug?
Answer: A `visited` set or closed-list tracking is maintained to record previously explored configurations, preventing infinite loops or circular transitions.

Q4: Which state space problems are best suited for DFS?
Answer: Problems with high branching factors, finite search depths, or multiple valid solution paths (e.g., Maze Solving, N-Queens, Sudoku solving), or where memory constraints restrict the usage of BFS.